

This document is a structural execution map and evaluation prompt template. Use this guide to audit, verify, restart, or troubleshoot the 'Engine\_1' algorithmic trading environment.

## ## 🔍 Codebase Directory Map

```
Path | Purpose | Key Verification Check |
| --- | --- | --- |
| [ /Engine_1.py ] [file:///C:/Users/SIGMA/Documents/Project%20-%20Class%20Trading/Engine_1/Engine_1.py] | Main allocation entry point & event loop. | Check that 'sys.stdout' uses UTF-8 reconfigure to prevent 'UnicodeEncodeError'. |
| [ /Engine_1_trade_logs.json ] [file:///C:/Users/SIGMA/Documents/Project%20-%20Class%20Trading/Engine_1/TradeLogs/Engine_1_trade_logs.json] | Persistent trade & order log. | Ensure JSON is valid and lists all operative positions. |
| [ /LiquidationTrain.py ] [file:///C:/Users/SIGMA/Documents/Project%20-%20Class%20Trading/Engine_1/LiquidationTrain.py] | ML Liquidation Walk-Forward Optimization (WFO). | Verify that the 14-day holdout evaluation is dynamically choosing the lookback window. |
| [ /alpha_squeeze_v17 ] [file:///C:/Users/SIGMA/Documents/Project%20-%20Class%20Trading/Engine_1/alpha_squeeze_v17] | AlphaSqueeze models, features, and config parameters. | Ensure symbols match configurations mapped in 'features.json'. |
```

## ## 🚀 CLI Operations (PowerShell Quick-Reference)

```
### 1. Verification of A ↓ Processes
Check if the engine or chromium is running in the background:
'''powershell
Get-Process -Name "python", "chrome" -ErrorAction SilentlyContinue |
Select-Object Id, ProcessName, CPU
'''
```

```
### 2. Clean Termination (Reset Environment)
Kill all active trading instances and browser contexts safely:
'''powershell
Stop-Process -Name "python", "chrome" -Force -ErrorAction SilentlyContinue
'''
```

```
### #3. Start Engine in persistent PopUp Terminal
Launch the engine in a persistent, visible shell (useful for viewing console
logs live):
'''powershell
Start-Process powershell -ArgumentList "-NoExit", "-Command", "python
Engine_1.py" -WorkingDirectory "C:\Users\SIGMA\Documents\Project -
Coinglass Trading\Engine_1"
'''
```

```
### 4. Read Live Trade Tracker Logs
powershell
Get-Content -Path "C:\Users\SIGMA\Documents\Project - Coinglass\
Trading\Engine_1\Engine_1_trade_logs.json" -Tail 20

```

## 📄 Copy-Paste Prompt for Future AI Review

When sharing this project with a new AI agent or session, copy and paste the block below to instantly bootstrap their understanding of your system:

```
"""text
Please perform a structural audit of our dual-strategy algorithmic trading
engine. Here is the operational context:
```

1. CODE BASE PATH: C:\Users\SIGMA\Documents\Project - Coinglass Trading\Engine\_1\Engine\_1.py
2. STRATEGY 1 (AlphaSqueezer V17): Uses a 3-model LightGBM/XGBoost/CatBoost ensemble voting approach. Features are dynamic. Config files are read from `alpha_squeezer_v17`.
3. STRATEGY 2 (ML Liquidity Runner): Uses Z-Scores to detect massive liquidations (>3.0 Z-score) on Coinglass. It passes 14 features into an LGBM/XGB/CB ensemble. It runs dynamic Walk-Forward Optimization (WFO) to choose lookback window (3, 6, 9 mo, or ALL) on a 20% out-of-sample test.
4. TRADE TRACKER: Engine1TradeTracker monitors both strategies concurrently, aggregates unrealized PnL, and enforces Blueberry Prop Firm rules (Athens timezone, 4% daily dd guardrail, 10% total dd).
5. UTF-8 console output is enforced at the `top/Engine_1.py` to prevent Windows' cp1252' charmap crashes.

- Check that there are no data leakage loops between the two strategies' queues.
- Verify that hot-swapping models during background training executes thread-safety (via RLock).
- Review Engine1TradeTracker's daily rollover logic for potential timezone offsets.

Extract the zip file first.

I'll extract the zip file and perform the structural audit

**Bash Command** `mkdir -p /home/user/engine && cd /home/user/engine && ls`

 [Download File](#) Engine\_1.zip → /home/user/engine

 File Downloaded

```

Filename: Engine_1.zip
Directory: /home/user/engine
Path: /home/user/engine/Engine_1.zip
Size: 27.79 MB
Source: https://www.genspark.ai/api/files/s/nmu0f094

```

```
Bash Command cd /home/user/engine && unzip -q Engine_1.zip -d Engine_1 extracted && echo "=="
```

```
=== Top-level structure ===
total 4
drwxr-xr-x 3 user user 60 Jul 13 15:41 .
...
Engine_1_extracted/Engine_1/chrome_profile/ShaderCache
Engine_1_extracted/Engine_1/chrome_profile/component_crx_cache
Engine_1_extracted/Engine_1/chrome_profile/extensions_crx_cache
```

```
Engine1_extracted/engine_1/chrome_profile/segmentation_platform
Exit Code: 0
```

```
Bash Command cd /home/user/engine/Engine_1_extracted/Engine_1 && echo "=== Root files ===" &... View

=== Root files ===
total 172
drwxr-xr-x 8 user user 4096 Jul 13 2026 .
...
./alpha_squeezer_v17/models/XAGUSD_long_xgb.json
./alpha_squeezer_v17/models/XAUUSD_short_xgb.json
./alpha_squeezer_v17/models/XRPUSD_long_xgb.json
./alpha_squeezer_v17/models/XRPUSD_short_xgb.json
Exit Code: 0
```

Extraction complete. Now let me get a size/line summary and then dive into the audit tasks you listed in the prompt.

```
Bash Command cd /home/user/engine/Engine_1_extracted/Engine_1 && echo "=== Line counts ===" &... View

=== Line counts ===
2993 Engine_1.py
231 Liquidation/features.py
...
=== Sizes ===
144K Engine_1.py
100K Liquidation
23M alpha_squeezer_v17
Exit Code: 0
```

Engine\_1.py is a 2,993-line monolith. Given the audit prompt (data-leakage between queues, RLock hot-swap safety, and timezone rollover in Engine1TradeTracker), I need to read the whole file targeted, not sequentially. Let me grep the critical surfaces first, then read the relevant regions.

```
Bash Command cd /home/user/engine/Engine_1_extracted/Engine_1 && echo "=== stdout / UTF-8 re... View

=== stdout / UTF-8 reconfigure (top of Engine_1.py) ===
# C:\Users\SIGMA\Documents\Project - CoinGlass Trading\Engine_1\Engine_1.py
# Production-Grade CoinGlass + Binance Footprint Scraper Terminal
...
1929:         rsl_val = parse_float(d.get("rsl", 0.0))
1931:         rsl_val = self.store_data.get(target_sym,
AssetSnapshot(symbol=target_sym)).rsl
1935:         volume=parse_float(d.get("volume", 0.0)),
1937:         fut_cvd=parse_float(d.get("futures_cvd", 0.0)),
Exit Code: 0
```

```
Bash Command cd /home/user/engine/Engine_1_extracted/Engine_1 && echo "=== UTF-8 lines arou... View

=== UTF-8 lines around top ===
import numpy as np
...
2588:async def renderer_loop(store: SnapshotStore, stop: asyncio.Event) -> None:
2617:async def watchdog(components: List[Any], focus_lock: asyncio.Lock, stop: asyncio.Event) -> None:
2651:def combine_seedfile_files4() -> None:
2720:async def main(skip_seed: bool = False) -> None:
Exit Code: 0
```

Good. Structure is clear. Now let me read the three critical audit surfaces: (1) LiveStrategyPredictor + LiveLiquidationPredictor (data-leakage / hot-swap RLock), (2) Engine1TradeTracker (Athens timezone rollover, DD guardrails), (3) the main loop wiring the two strategies together.

```
Read File /home/user/engine/Engine_1_extracted/Engine_1/Engine_1.py View

196 class LiveStrategyPredictor:
197     def __init__(self, symbols: List[str]):
198         self.symbols = symbols
...
692
693     def check_model_updates(self):
694         """Checks if model files were updated by the WFO training script and hot-swaps them."""
695         base_dir = os.path.dirname(os.path.abspath(__file__))
```

```
Read File /home/user/engine/Engine_1_extracted/Engine_1/Engine_1.py View

693     def check_model_updates(self):
694         """Checks if model files were updated by the WFO training script and hot-swaps them."""
695         base_dir = os.path.dirname(os.path.abspath(__file__))
...
989         import zoneinfo
990         from datetime import datetime
991         broker_tz = zoneinfo.ZoneInfo("Europe/Athens")
992
```

Continuing into Engine1TradeTracker — the timezone rollover logic is the key audit surface.

```
Read File /home/user/engine/Engine_1_extracted/Engine_1/Engine_1.py View

985     }
986     self.lock = threading.RLock()
987     self.load_history()
...
1336
1337 class SnapshotStore:
1338     def __init__(self, symbols: List[str], predictor: LiveStrategyPredictor = None,
liquidation_predictor: Any = ...
1339         self_data: Dict[str, AssetSnapshot] = {s: AssetSnapshot(symbol=s) for s in symbols}
```

Excellent — I now have full visibility into the three critical audit surfaces. Let me also check where update\_day is invoked from (rollover trigger) and check the WFO trainer for thread-safety of the model-swap.

```
Bash Command cd /home/user/engine/Engine_1_extracted/Engine_1 && echo "=== Who invokes up... View

=== Who invokes update_day? ===
1047: def update_day(self) -> None:
1372:         self.trade_tracker.update_day()
...
    "NATGASUSD": 0.0002, # Tick size for Natural Gas
}

class FootprintCandle:
```

Exit Code: 0

Now I have enough on the wiring. Let me also check the Liquidation train.py for the WFO daemon thread, model-swap contract, and any leakage possibility.

Read File /home/user/engine/Engine\_1\_extracted/Engine\_1/Liquidation/train.py View

```
1 import os
2 import polars as pl
3 import numpy as np
...
200 print("All models trained and saved to models/ successfully!")
201
202 if __name__ == "__main__":
203     train_ensemble()
```

Now let me confirm how the WFO daemon thread is wired from Engine\_1.py (the audit prompt claims "24-hour daemon thread"):

Bash Command cd /home/user/engine/Engine\_1\_extracted/Engine\_1 && echo "=== All Thread(). dae... View

```
=== All Thread(), daemon=, subprocess, and train mentions ===
693: def check_model_updates(self):
694:     """Checks if model files were updated by the WFO training script and hot-swaps them."""
...
2784:     time.sleep(10000)
2793:     from train import train_ensemble
2794:     train_ensemble()
2800:     retrain_thread = threading.Thread(target=background_retrain_loop, daemon=True)
```

Exit Code: 0

Read File /home/user/engine/Engine\_1\_extracted/Engine\_1/Engine\_1.py View

```
2740 print("[Setup] Running Live Model Retraining on latest Parquet data...")
2741 try:
2742     as_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'alpha_squeezer_v17')
...
2856 print("[Setup] Submitting login form...")
2857
2858 # Try explicit button click first, fallback to JS click, last resort enter key
2859 try:
```

I now have complete visibility over the audit surfaces. I also spotted a **CRITICAL** issue — hardcoded credentials on lines 2843–2844. Let me do a couple of small confirmations before writing the report.

Bash Command cd /home/user/engine/Engine\_1\_extracted/Engine\_1 && echo "=== active\_trades re... View

```
=== active_trades read/write outside self.lock ===
633: if armed_str and trade_tracker and not any(t['symbol'] == symbol and t['strategy'] ==
634: "AlphaSqueezer...
668: trades = [t for t in trade_tracker.active_trades.values() if t['symbol'] == symbol and
669: t['strategy'] == ...
...
657: trade_tracker.last_entry_bar[symbol] = last_bar_time
978: self.last_entry_bar = {}
1012: self.last_entry_bar = meta.get('last_entry_bar', {})
1036: 'last_entry_bar': dict(self.last_entry_bar)
```

Exit Code: 0

I have everything needed. Extraction is complete and I've read every file relevant to the audit. Here is the Principal-Engineer review.

#### Engine\_1 — Structural Audit Report

**Repo:** Engine\_1/ (2,993-line monolith + Liquidation/, alpha\_squeezer\_v17/) **Log state loaded:** trade tracker JSON is valid, entries have exit prices and Athens-TZ-stamped ISO times — envelope format intact. **UTF-8 reconfig** (Engine\_1.py:30–37): ☒ present, hasattr guarded, wrapped in try/except — correct.

#### 1. Data-Leakage Between the Two Strategies' Queues

**Verdict:** ▲ **No queues, but there are two live cross-strategy coupling risks — one benign, one silent.**

There is no queue.Queue in the code. Both strategies pull from the same SnapshotStore.update() funnel (Engine\_1.py:1346), which invokes each predictor's on\_tick\_update() on asyncio.to\_thread in sequence. So the audit prompt's "queues" are really the two predictors' internal candles\_history dequeues. Two coupling points exist:

##### A. Shared candle warm-up on startup (Engine\_1.py:2766–2772) — SAFE.

liquidation\_predictor.candles\_history is seeded from predictor.candles\_history via copy.deepcopy. Deque objects and dict rows are independently owned — no aliasing. ☒ This is correctly handled and the code comment even flags the deepcopy intent.

##### B. Post-warmup, the two predictors *independently* rebuild candles from the same live

AssetSnapshot — **SAFE**. Each predictor owns its own current\_candle / candles\_history and updates them from the snapshot passed in (Engine\_1.py:466–495 and 747–782). The snapshots are frozen dataclasses (AssetSnapshot) built inside SnapshotStore.update() via dataclasses.replace(...) — immutable, so no back-propagation into either strategy's history is possible.

##### C. ▲ **Actual leakage risk** — trade\_tracker.active\_trades is read *outside* trade\_tracker.lock from LiveStrategyPredictor at lines 633 and 668.

```
# Line 633 - AlphaSqueezer predictor, no lock held:
if armed_str and trade_tracker and not any(
    t['symbol'] == symbol and t['strategy'] == "AlphaSqueezer_v17"
    for t in trade_tracker.active_trades.values()
) and atr_val > 0:
```

Copy

Engine1TradeTracker.check\_exits() (line 1304) does del self.active\_trades[...] inside its RLock. When the two coroutines interleave — check\_exits on thread A, LiveStrategyPredictor.on\_tick\_update on thread B — thread B can hit RuntimeError: dictionary changed size during iteration on .values(). Not data leakage in the ML sense, but it corrupts the "no duplicate entry" guard: a rare race where thread B reads active\_trades between A's exit and A's save\_history() can double-open the same position within the SL/TP cooldown window.

**Fix:** wrap the read side in with trade\_tracker.lock: or expose a snapshot\_active() method that returns a copy under the lock. Same fix at 2536.

#### 2. Thread-Safe Hot-Swap During Background Training (RLock)

**Verdict:** ✗ **Hot-swap is NOT thread-safe. This is the highest-severity finding.**

The intended contract per the audit prompt is: WFO on a 24-hour daemon thread rewrites `Liquidation/models/*.pkl`, and `LiveLiquidationPredictor.check_model_updates()` detects the mtime bump and reloads. Two independent defects:

**A. Retraining runs *in-process*, not out-of-process** (Engine\_1.py:2780-2802).

```
def background_retrain_loop():
    while True:
        time.sleep(60*60)
        from model_trainer import train_models  # AlphaSqueezer
        train_models()
        from train import train_ensemble        # Liquidation
        train_ensemble()

    retrain_thread = threading.Thread(target=background_retrain_loop, daemon=True)
    retrain_thread.start()
```

`train.py:150,161,172` fit `LightGBM/XGBoost/CalBoost` — all release the GIL for long stretches and can pin cores at 100%. During that window, the live inference loop starves and Playwright websocket keepalives can drop. Retraining should be a subprocess (`multiprocessing.Process`) not a daemon `Thread`.

**B. The RLock does not protect the swap** — `load_models()` is called under `_lock`, but with only *partial* file-set validation.

```
# Engine_1.py:693-702
def check_model_updates(self):
    lgb_path = os.path.join(base_dir, "Liquidation", "models", "lgb_model.pkl")
    if os.path.exists(lgb_path):
        mtime = os.path.getmtime(lgb_path)
        if mtime > self.last_model_mtime:
            print(f"[Liquidation] Detected new WFO ML models (mtime: {mtime})...")
            self.load_models()
            self.last_model_mtime = mtime
```

And `check_model_updates` is called from inside `on_tick_update` (line 737), which is already inside with `self._lock`: That prevents *inference* from racing the swap. But the trainer writes the three pickles **one at a time** and **non-atomically** (`train.py:175-180`):

```
with open(os.path.join(models_dir, "lgb_model.pkl"), "wb") as f: pickle.dump(lgb_model, f)
with open(os.path.join(models_dir, "xgb_model.pkl"), "wb") as f: pickle.dump(xgb_model, f)
with open(os.path.join(models_dir, "cb_model.pkl"), "wb") as f: pickle.dump(cb_model, f)
```

If the predictor thread checks `lgb_model.pkl`'s mtime *between* the LGB write and the XGB/CB writes, `load_models()` (line 704-730) will pickle-load a fresh LGB, then a **stale** XGB and CB from the *previous* WFO window. The class-mapping assertion at line 726 will pass (all models still expect classes `[0,1,2]`), so the mismatch is silent — you'll run inference with a fresh LGB voting alongside a stale XGB/CB for up to ~2 minutes.

**Fix (atomic swap pattern):**

1. In `train.py`, write to `.pkl.tmp` then `os.replace()` to final name for each of the three; or better, write all three under a `models_new/` staging dir and `os.replace()` the directory.
2. In `check_model_updates`, compute the *max* mtime across all three files and require all three to be newer than `last_model_mtime` before triggering the swap.
3. Ship a `manifest.json` last (with SHA-256 of all three), and gate the reload on manifest freshness — this is the only file the predictor should mtime-poll.

**C. `sys.path.insert(0, liq_dir)` on every reload** (Engine\_1.py:711). Every hot-swap re-inserts the same directory into `sys.path`, which is not thread-safe when the retrain thread is simultaneously doing from `features` import ... Guard with if `liq_dir` not in `sys.path` — actually already done — but the from `features` import `compute_rolling_features` re-import at line 714 will happily pull a cached, stale module. Force a fresh import via `importlib.reload(features_module)` if `features.py` was rewritten.

**3. Engine1TradeTracker Daily Rollover Logic — Athens Timezone Offset**

**Verdict:** ▲ **Timezone handling is correct; the rollover trigger is broken.**

**A. Timezone math itself is fine** (Engine\_1.py:990-993, 1047-1061).

`zoneinfo.ZoneInfo("Europe/Athens")` correctly handles Athens DST (last Sunday of March / October) — no naive offset arithmetic. `datetime.now(broker_tz).strftime("%Y-%m-%d")` is the right primitive.

✓ On the DST spring-forward night, the local date string still increments cleanly at Athens midnight, which is what Blueberry Prop Firm's daily DD rule uses.

**B. Rollover is triggered only on price ticks** (`SnapshotStore.update`, line 1372).

```
if self.trade_tracker:
    self.trade_tracker.update_day()
```

`update_day()` is only ever called from inside a websocket price-update coroutine. Consequence: **if no symbol ticks for the entire minute containing Athens midnight** (e.g. commodity feeds like `XAUUSD`, `NATGASUSD`, `CLUSD` between 23:00 UTC Friday and 22:00 UTC Sunday), `last_rollover_day` never advances and `daily_start_capital` stays anchored to the previous session's equity. On Monday-morning resumption, the first tick fires the rollover — but *the daily DD guardrail was silently misaligned for the entire weekend*.

Since the log shows commodity symbols (`CLUSD`, `XAUUSD`, `XAGUSD`, `NATGASUSD`) that trade on CME hours, this window is real, not theoretical.

**Fix:** Add a dedicated async task in `main()` that awakens every 60 seconds and calls

`trade_tracker.update_day()`:

```
async def rollover_watchdog(tracker, stop):
    while not stop.is_set():
        tracker.update_day()
        await asyncio.sleep(60)
```

Attach it alongside `renderer_loop/watchdog` at ~2960.

**C. `daily_start_capital = current_equity` (line 1059) includes unrealized PnL**. This is *arguably* correct for Blueberry Prop's Athens-close-mark-to-market rule, but if their contract instead uses realized-only equity at 00:00 Athens, this bakes in a systematic bias equal to whatever unrealized book existed at rollover. Verify against the current Blueberry Prop Firm rulebook — either behavior is defensible, but it must match the exact wording.

**D. `total_dd denominator` is hardcoded to \$5,000** (Engine\_1.py:1081, 1186).

```
total_dd = (5000.0 - current_equity) / 5000.0
```

If `initial_capital` is ever re-configured through `EngineTradeTracker(initial_capital=...)`, the total-DD guard breaks silently. Use `self.initial_capital` for both the numerator baseline and denominator.

Secondary Findings (in decreasing severity)

- 1. **Hardcoded credentials in source** (Engine\_1.py:2843-2844). Coinglass email and password are committed as string literals. Rotate the password immediately and move both to `os.environ` with no default. Given this is git-tracked in a Coinglass Trading project, treat the current credential as *already compromised*.
- 2. **update\_live\_pnl emergency-halt liquidates only trades on the ticking symbol** (line 1197). When the halt fires because BTCUSD1T drew down through the -4.5% daily guardrail, only BTC positions get liquidated — ETHUSD1T, SOLUSD1T, etc. remain open until each of those symbols ticks. This is exactly the wrong direction: the halt is triggered by aggregate DD but only closes one symbol at a time, extending exposure during the very drawdown you're trying to stop. Iterate over *all* active\_trades and mark them with `pending_emergency_exit=True`; close each on the first tick per-symbol using its own current price.
- 3. **daily\_start\_capital computed before load\_history() finishes replaying old trades** — actually it's fine (line 987 → 988 order) but the current design means a restart mid-session resets the "daily start" to the reloaded capital, effectively giving the account a fresh 4% guardrail if the process restarts after a partial-day drawdown. Persist `daily_start_capital` and `last_rollover_day` in the log envelope's `__meta__` block alongside `last_entry_bar`.
- 4. **load\_history() filter** (Engine\_1.py:1815) **drops any trade without exit\_price** — correct, active trades handled separately — but the string `'test'` not in `trade_id.lower()` will also drop legitimate trades on any symbol containing "test" in a future listing (unlikely for crypto/commodities but worth a stricter marker like a leading TEST\_).
- 5. **renderer\_loop reads trade\_tracker.active\_trades.values() (line 2536) with no lock** — same issue as Finding 1C, but from the Rich Live renderer thread. During a `del self.active_trades[...]` in `check_exits`, the renderer can raise; caught silently by the rich library, but you'll see flickering/gaps in the terminal table.
- 6. **train.py uses X\_test = X\_train (line 128)** — "dummy test for logging" per the comment. Confirmed intentional but the printed accuracy is meaningless in-sample. The actual OOS metric already lives in the 14-day window loop (line 109). Consider explicitly logging `# TRAINING accuracy` — not `OOS` to avoid future confusion.
- 7. **AlphaSqueezer prediction has no equivalent to check\_model\_updates**. Its `load_models()` is called only in `__init__`. When `background_retrain_loop` rewrites AlphaSqueezer models at 24 h, they are **never picked up** until the process restarts — the "hot-swap" applies to Liquidation only. Confirm this is intentional, if not, add `check_model_updates` parity to `LiveStrategyPredictor`.

Priority Fix Order

| # | Severity | Fix                                                             | Location                                          |
|---|----------|-----------------------------------------------------------------|---------------------------------------------------|
| 1 | P0       | Rotate Coinglass credential, remove from source                 | Engine_1.py:2843-2844                             |
| 2 | P0       | Atomic 3-file model swap + manifest                             | Liquidation/train.py:175-180, Engine_1.py:693-730 |
| 3 | P1       | Rollover watchdog task independent of ticks                     | Engine_1.py:2720-2960 main()                      |
| 4 | P1       | Emergency halt closes ALL symbols, not just ticking one         | Engine_1.py:1194-1220                             |
| 5 | P2       | Guard trade_tracker.active_trades reads outside its lock        | Engine_1.py:633, 668, 2536                        |
| 6 | P2       | Move retraining to multiprocessing.Process (release GIL)        | Engine_1.py:2780-2802                             |
| 7 | P3       | AlphaSqueezer hot-swap parity                                   | Engine_1.py:196-290                               |
| 8 | P3       | Persist daily_start_capital + last_rollover_day in log envelope | Engine_1.py:1030-1045                             |

If you want, I can produce a single-file patch diff for the P0/P1 items (atomic model swap + rollover watchdog + credential purge) — say the word and I'll draft it.



Super Agent   •   Genspark vs ChatGPT vs Claude →

Ask anything, create anything

+

Ultra ▾

Connectors

🗣️ Speak